

RAVEN — Avaliação Didática

ddnr03@gmail.com [Mudar de conta](#)



✉ Não compartilhado

* Indica uma pergunta obrigatória

Seção 3 — Atividade Guiada no RAVEN (≈55 min)

Sobre o RAVEN: é um simulador educacional de um processador RISC-V (arquitetura de conjunto de instruções aberta e reduzida) com visualização de pipeline, hierarquia de cache e execução multi-core.

Download: baixe o executável para a sua arquitetura em <https://github.com/Gaok1/Raven-RiscV/releases/tag/v1.27.6> e execute o aplicativo.

Como funcionam as atividades: cada pergunta abaixo indica um preset identificado por um código (ex.: D1-01, D3-05). Na aba "Activity" do RAVEN, selecione o preset indicado. O simulador carrega automaticamente o programa e as configurações necessárias.

Atenção: Antes de iniciar as atividades, leia o tutorial de cada aba que estiver ativa. Você pode abrir esse tutorial pressionando a tecla "h" ou clicando no botão "[?]" da aba respectiva.

Nesta seção, o importante é observar o comportamento do simulador e registrar o que você viu.

Em geral, você vai alternar entre estas abas:

- Run, para acompanhar execução, CPI (Ciclos Por Instrução), registradores e detalhes das instruções.
- Pipeline, para observar stalls (esperas), flushes (descartes de instruções) e o avanço por ciclo.
- Cache, para comparar miss rate (taxa de falhas), AMAT (Tempo Médio de Acesso à Memória) e diferenças entre configurações.

Orientações gerais:

- Execute o preset até o final e anote os valores solicitados.
- Em campos numéricos, informe apenas o valor observado (use ponto ou vírgula para decimais).
- Em observações visuais, foque apenas no trecho ou momento indicado.
- As respostas descritivas valem pela observação, não pelo acerto.

D1 — Pipeline (≈12 min)

O pipeline surgiu para aumentar o aproveitamento do processador. Sem pipeline, uma instrução precisa terminar todo o seu caminho interno antes que a próxima realmente comece a usar o hardware. Isso funciona, mas deixa partes do processador ociosas em vários momentos.

A ideia do pipeline é dividir a execução em etapas menores e fazer com que instruções diferentes ocupem etapas diferentes ao mesmo tempo. Assim, enquanto uma instrução está mais adiante, a seguinte já pode começar a etapa inicial, em vez de esperar a primeira terminar por completo.

No modelo clássico de 5 estágios desta arquitetura:

- IF (Busca da Instrução)
- ID (Decodificação da Instrução)
- EX (Execução)
- MEM (Acesso à Memória)
- WB (Escrita de Volta)

Cada instrução ainda precisa passar por todas essas etapas. O que muda é que, em vez de uma única instrução ocupar sozinha todo o processador do começo ao fim, várias instruções caminham em paralelo, cada uma em um estágio diferente.

Esse paralelismo melhora o fluxo de execução, mas traz hazards (conflitos). Eles aparecem quando a próxima instrução precisa de um valor, de um recurso ou de uma decisão que ainda não ficou pronta.

O caso mais importante aqui é RAW (Leitura Após Escrita): uma instrução tenta ler um valor que a anterior ainda não terminou de produzir. Quando possível, o processador usa forwarding (encaminhamento) para repassar o resultado diretamente entre estágios. Quando isso não basta, surgem bolhas ou stalls (ciclos de espera), que atrasam o avanço normal do pipeline.

CPI com instruções independentes *

Preset: D1-01.

Este preset usa um programa com instruções independentes e pipeline habilitado, sem dependências fortes entre instruções consecutivas.

Execute o programa até o final e anote o CPI (Ciclos Por Instrução) exibido na aba Run. utilize as seguintes hotkeys na aba Run:

- **p** : para pausar ou executar
- **f** : para alternar a **velocidade**
- **s**: para avançar apenas 1 ciclo/instrução depende da aba atual. Run: instrução, Pipeline: ciclo

CPI = total de ciclos ÷ total de instruções executadas.

Sua resposta



CPI com instruções dependentes (cada instrução usa o resultado da anterior) *

Preset: D1-02.
Este preset usa uma cadeia de dependências RAW (Leitura Após Escrita) com forwarding (encaminhamento) desativado, para tornar os stalls visíveis.

Execute o programa e anote o CPI (Ciclos Por Instrução) exibido na aba Run.
Compare com o valor obtido no D1-01.

Sua resposta

O que o pipeline realmente melhora? *

Compare agora os dois presets usando a aba **Pipeline**.

Primeiro, carregue o **Preset D1-01** e avance alguns ciclos (**s**) na aba **Pipeline**. Observe como as instruções independentes se movem pelos estágios do pipeline e se aparecem bolhas/stalls no diagrama.

Bolhas são ciclos vazios inseridos pelo pipeline entre instruções, aparecem como células em branco ('—').

Depois, carregue o **Preset D1-02** e avance alguns ciclos na aba **Pipeline**. Observe como as dependências RAW (Read After Write, Leitura Após Escrita) afetam o avanço das instruções, principalmente com forwarding (encaminhamento) desativado.

O que muda entre o D1-01 e o D1-02 na movimentação das instruções pelo pipeline? Qual deles apresenta mais bolhas/stalls? Explique por que isso acontece.

Sua resposta

D2 — Hazards de Dados e Controle (≈15 min)

Nem todo problema no pipeline tem a mesma causa.

Hazards de dados:

- acontecem quando uma instrução precisa usar um valor que ainda não ficou pronto
- um caso clássico é o load-use hazard, em que o valor lido por um load chega tarde demais para a instrução seguinte

Hazards de controle:

- acontecem quando o processador segue um caminho e depois descobre que deveria ter ido por outro
- isso aparece em branches e jumps
- quando isso ocorre, pode haver flush (descarte) das instruções buscadas no caminho errado

Nesta parte, observe a diferença entre stall (espera por dado) e flush (correção do fluxo de controle).

Load-use: bolha automática no par lw/add *

Preset: D2-01.
Este preset mostra um caso clássico de load-use hazard, em que uma instrução usa logo em seguida o valor carregado por um lw.

Avance ciclo a ciclo na aba Pipeline até a primeira ocorrência do par lw/add no diagrama. prossiga até ver a instrução add sair do estágio de EX

utilize a keyword **s** para **skipar** 1 ciclo de execução.

Quantas bolhas aparecem antes do add entrar em execução nessa primeira ocorrência?

Sua resposta

Load-use: o que muda ao inserir um nop entre lw e add *

Mantenha o Preset D2-01 já carregado.
Aqui, a ideia é comparar a bolha inserida automaticamente pelo hardware com uma separação manual feita por você no código.

nop é uma instrução que não produz efeito útil no estado do programa; aqui ela serve apenas para criar distância entre duas instruções dependentes.

Na aba **Editor**, localize as linhas lw t1, 0(t0) e add t2, t1, t1 e insira uma linha "nop" entre elas.
Pressione F5 para recompilar.

Atenção: A instrução **nop** é uma pseudoinstrução que é traduzida para (addi zero,zero,0). Não confundir no pipeline com a instrução 'add' que deve ser observada (add t2, t1, t1)

Acesse a aba **pipeline** novamente e então:
Execute e observe até ver a instrução add sair do estágio de EX.

Quantas bolhas aparecem antes do add agora? Compare com o resultado anterior.

Sua resposta



Diferença visual entre stall e flush no diagrama de pipeline *

Use **D2-02** (Utiliza a heurística que assume que desvios não serão tomados) para observar um flush por desvio de controle. Avance até o primeiro **beq** e observe o que acontece com as instruções anteriores.

Qual a relação do **flush** com base na heurística do preset?

Sua resposta

D3 — Cache (≈10 min)

A cache é uma memória pequena e rápida que tenta evitar acessos frequentes à memória principal.

A cache ganha desempenho explorando dois tipos de localidade:

- **Temporal**: o que foi usado recentemente tende a ser usado novamente.
- **Espacial**: o que está próximo na memória tende a ser acessado em sequência.

Quando o dado já está na cache, ocorre hit (acerto).

Quando não está, ocorre miss (falha).

Na visualização da cache:

- **Set**: grupo lógico da cache onde um bloco de memória pode ser colocado, definido por parte do endereço.
- **Way**: uma das posições possíveis dentro de um set, usada em caches associativas.

Uma métrica central aqui é AMAT (Tempo Médio de Acesso à Memória), que combina o custo dos acertos com o custo das falhas.

- **Atenção**: A cache observada deve ser a **D-cache** (Cache de dados).

Também importam:

- a política de substituição, como LRU (Least Recently Used, Menos Recentemente Usado) e FIFO (First In, First Out, Primeiro a Entrar, Primeiro a Sair)

- os conflitos de mapeamento, que podem causar **Thrashing**: descarte excessivo de dados da cache, causando muitos cache misses.

AMAT — Config A *

Preset: D3-01.

Este preset usa uma D-Cache pequena, com hit time baixo, em um programa cujo conjunto de dados não cabe inteiro nela.

Execute o programa e leia o AMAT da D-Cache no resumo da aba Cache.

Esse valor representa o custo médio de acesso à memória, combinando acertos e falhas.

Informe o valor observado (em ciclos).

Sua resposta

⚠ Esta pergunta é obrigatória

AMAT — Config B *

Preset: D3-02.

Este preset mantém o mesmo programa, mas troca para uma D-Cache maior, que comporta melhor o conjunto de dados, com hit time mais alto.

Execute e leia o AMAT da D-Cache na aba Cache.

Lembre: um AMAT menor indica acesso médio mais eficiente.

Informe o valor observado.

Sua resposta

Com base no AMAT observado, qual configuração apresentou melhor desempenho de acesso à memória? *

Use os dois valores que você mediu acima.

- ☐ Config A
- ☐ Config B
- ☐ Empate / muito semelhante

Miss rate com LRU em varredura sequencial *

Preset: D3-03.

Este preset usa acesso sequencial à memória com política LRU.

Execute e leia o miss rate (taxa de falhas) da D-Cache no resumo da aba Cache.

Miss rate é a fração de acessos que não encontraram o dado na cache.

Hit rate é a fração de acessos que encontraram.

Informe o miss rate observado (entre 0 e 1, ou porcentagem).

Sua resposta



Miss rate com FIFO em varredura sequencial *

Preset: D3-04.
Este preset mantém o mesmo acesso sequencial, mas troca a política de substituição para FIFO (First In, First Out, Primeiro a Entrar, Primeiro a Sair).

Execute e leia o miss rate (taxa de falhas) da D-Cache (Cache de Dados) na aba Cache. Compare diretamente com o valor anterior: quanto menor o miss rate, menos vezes a cache falhou.

Informe o miss rate e compare com o resultado do LRU acima.

Sua resposta

LRU vs. FIFO em varredura sequencial — por que o resultado é (quase) o mesmo? *

Compare os miss rates (taxas de falha) de LRU e FIFO com D301.

É recomendado acessar a aba **Editor** e verificar o código do programa para identificar o padrão de acesso.

O que os dois valores sugerem sobre como o programa acessa a memória?
Em que tipo de situação você esperaria que LRU e FIFO produzissem resultados diferentes?

Sua resposta

Thrashing por Conflito em Cache 2-Way *

Preset: D3-05.
Este preset executa um caso de thrashing por conflito com cache 2-way, em que vários blocos competem pelo mesmo conjunto e a associatividade disponível não é suficiente para mantê-los juntos.

Execute e leia o miss rate (taxa de falhas) da D-Cache (Cache de Dados) na aba Cache. Compare diretamente com o valor anterior: quanto menor o miss rate, menos vezes a cache falhou.

Informe o miss rate e compare com o resultado do LRU acima.

Sua resposta

D4 — ISA (Arquitetura do Conjunto de Instruções) do RISC-V (≈8 min)

A ISA (Arquitetura do Conjunto de Instruções) define como as instruções são representadas e interpretadas pelo processador.

No RISC-V (arquitetura de conjunto de instruções aberta e reduzida), formatos como R-type, I-type e S-type distribuem campos como:

- registradores
- operação
- imediato

Essa organização não é arbitrária: ela facilita a decodificação e o funcionamento do hardware.

Nesta parte, a ideia é perceber que a codificação binária da instrução está ligada diretamente ao que o processador consegue executar.

Codificação binária: por que S-type não tem campo rd? *

Preset: D4-01.
Este preset mostra um trecho curto com uma instrução do tipo R-type e outra do tipo S-type.
Para esta atividade, utilize a aba **Run** e identifique na janela à direita **Instruction, Field Map** e **Decoded**

Essas abas possibilitam **visualizar** a instrução de forma decodificada e a estudar como cada tipo de instrução é codificado.

Para operar de forma melhor, clique nas instruções pela janela **Instruction Memory**.

Atenção:
A notação [n:m] indica um intervalo inclusivo de bits de um campo.

Exemplo:
imm[11:5] e imm[4:0]
Embora o imediato apareça dividido em dois campos na instrução, ele representa um único valor. A parte *imm[4:0]* contém os 5 bits menos significativos, enquanto *imm[11:5]* contém os 7 bits seguintes.

Juntas, essas duas partes formam um imediato de 12 bits:
imm[11:0] = imm[11:5] + imm[4:0]

Na aba Run, mova o cursor sobre a instrução de tipo **[R]** e depois sobre a instrução **[S]**. O painel de detalhes à direita mostra a codificação binária de cada uma.

Quais campos de decodificação novos surgem? Quais de **[R]** foram substituídos? Anote o tamanho em bits de cada campo novo/substituído.

Realize as mesmas anotações entre uma instrução de tipo **[S]** e tipo **[I]**.

Sua resposta



D5 — Hardware threads e multi-core (≈8 min)

Em um sistema multi-core, mais de um núcleo pode executar instruções ao mesmo tempo.

Cada core mantém:

- seu próprio fluxo de execução
- seu próprio banco de registradores

No contexto desta atividade, pense em cada core ou hart como uma unidade independente de execução que o simulador permite inspecionar separadamente.

Ao mesmo tempo, os cores podem compartilhar memória e outros recursos do sistema.

Por isso, é importante distinguir o que é estado privado de cada core e o que é estado compartilhado.

Nesta parte, observe como dois fluxos de execução coexistem sem misturar automaticamente seus registradores.

Atenção!

Este simulador utiliza o conceito de HARTs (Hardware Threads), isto é, fluxos de execução em hardware.

Diferente de uma thread comum, que pode ser escalonada dinamicamente pelo SO (Sistema Operacional), uma HART é associada a um núcleo de execução do simulador e executa seu programa até o fim, sem escalonamento pelo sistema operacional.

Com dois cores executando programas diferentes, ao inspecionar seus bancos de registradores, os valores de x1, x5 e o PC são: *

Preset: D5-01.

Este preset executa dois cores ao mesmo tempo, com valores diferentes nos registradores, para destacar o que é privado de cada core e o que é compartilhado.

Execute o programa por alguns ciclos.

Na aba Run, troque entre Core 0 e Core 1 usando o seletor de core.

Observe x1, x5 e o PC (Contador de Programa) em cada core.

- ☐ Idênticos nos dois cores, pois compartilham o mesmo banco de registradores
- ☐ Diferentes, pois cada core possui seu próprio banco de registradores privado, completamente independente do outro
- ☐ Idênticos para registradores de dados (x1–x31), mas com PC diferente em cada core
- ☐ Diferentes apenas enquanto os programas executam; sincronizam ao final da execução

O que Core 0 e Core 1 compartilham, e o que é privado em cada um? *

Com os dois cores executando, compare o que você vê ao trocar entre eles:

- Os registradores mudam ao trocar de core?
- A visão da RAM (Memória de Acesso Aleatório) muda?

Dica: se um valor pertence ao banco de registradores ou ao fluxo de execução de um core, ele tende a ser privado. Se pertence ao espaço de memória acessado por ambos, ele tende a ser compartilhado.

Descreva o que parece ser individual de cada core e o que parece ser comum aos dois.

Sua resposta

D6 — Métricas de Desempenho (≈7 min)

Desempenho não depende apenas de “ter pipeline”, mas de quanto paralelismo real ele consegue sustentar.

Métricas centrais nesta parte:

- CPI (Ciclos Por Instrução): quantos ciclos, em média, cada instrução consumiu
- speedup: quanto uma configuração ficou mais rápida em relação a outra

Se houver stalls, flushes, dependências de dados ou atrasos de memória, o ganho real fica abaixo do ideal teórico.

Aqui, a meta é relacionar o comportamento observado no simulador com os números medidos.

CPI sem pipeline *

Preset: D6-01.

Este preset executa a mesma cadeia de dependências do D102, mas com o pipeline desabilitado, servindo como referência sequencial.

Execute o programa e anote o CPI (Ciclos Por Instrução) exibido na aba Run.

Sua resposta



CPI com pipeline habilitado *

Preset: D6-02.

Este preset executa o mesmo programa do item anterior, agora com o pipeline habilitado, para permitir comparar o ganho real de desempenho.

Execute o mesmo programa e anote o CPI (Ciclos Por Instrução).

Sua resposta

Speedup observado (CPI sem pipeline ÷ CPI com pipeline) *

Speedup = CPI_sem ÷ CPI_com.

Se o resultado for maior que 1, a segunda configuração foi mais eficiente.

Calcule com os dois valores acima e informe o resultado.

Sua resposta

⚠ Esta pergunta é obrigatória

Por que o speedup ficou abaixo do esperado? *

Com base no que você observou ao longo da sessão (pipeline, caches):
o que pode ter impedido o pipeline de atingir o speedup máximo possível?

Justifique com base no que você viu no RAVEN.

Sua resposta

Voltar

Avançar

Limpar formulário

Nunca envie senhas pelo Google Formulários.

Este conteúdo não foi criado nem aprovado pelo Google. - [Entre em contato com o proprietário do formulário](#) - [Termos de Serviço](#) - [Política de Privacidade](#)

Este formulário parece suspeito? [Denunciar](#)

Google Formulários



